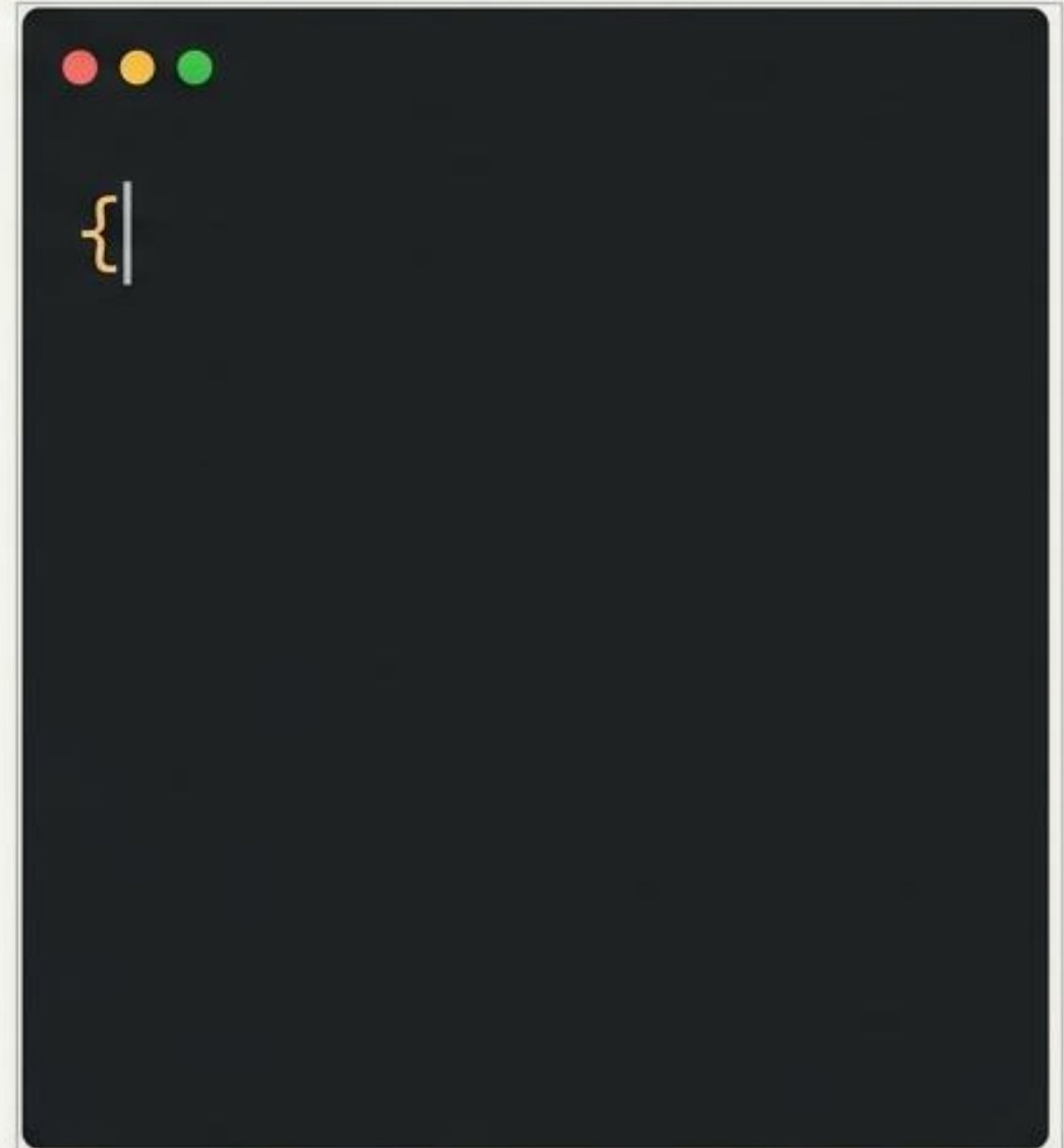
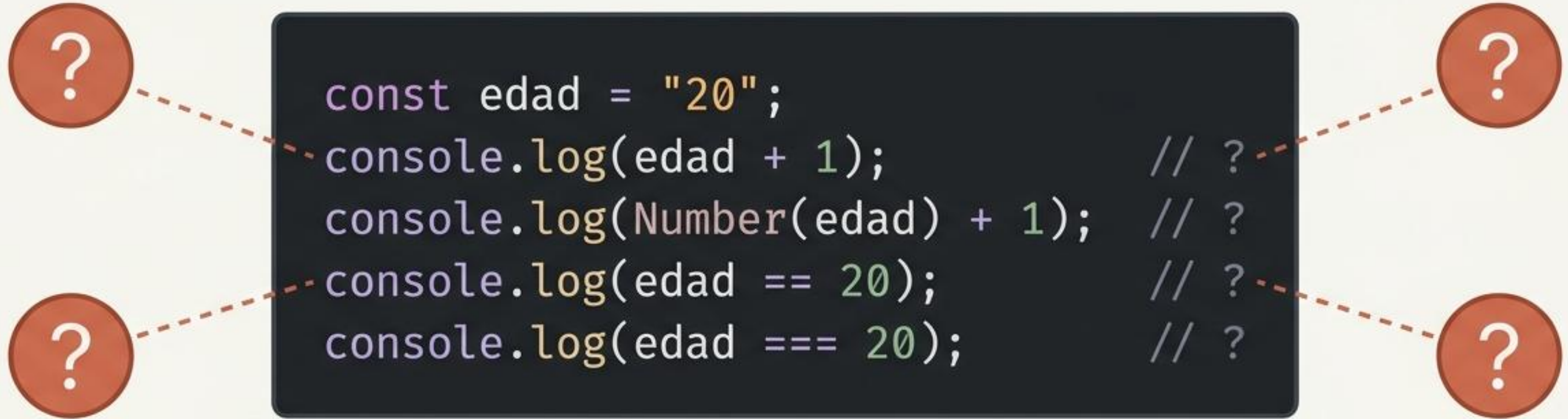


JavaScript Moderno: Fundamentos y Predictibilidad

Variables, Tipos, Coerción y el
puente hacia React.



El Misterio de la Consola



```
const edad = "20";  
console.log(edad + 1);           // ?  
console.log(Number(edad) + 1);  // ?  
console.log(edad == 20);        // ?  
console.log(edad === 20);       // ?
```



¿Por qué JavaScript hace algo aparentemente distinto en cada caso? Antes de avanzar, predice el resultado mentalmente.

El Misterio de la Consola

```
const edad = "20";  
console.log(edad + 1);           // 201  
console.log(Number(edad) + 1);  // 21  
console.log(edad == 20);        // true  
console.log(edad === 20);       // false
```

Coerción implícita: JS convirtió el 1 en texto y los concatenó.

Conversión explícita: Forzamos el cambio del texto a número.

Igualdad débil (==): Compara el valor, ignorando el tipo.

Igualdad estricta (===): Compara valor absoluto Y tipo.

Para dominar React, necesitamos reemplazar las suposiciones del lenguaje clásico por control absoluto.

El Nuevo Estándar: Variables



let

Permite declarar una variable cuyo valor puede cambiar.

```
let puntos = 10;  
puntos = 15; // ✓ 15
```



const

Declara una referencia que no puede ser reasignada.

```
const nombre = "Ana";  
nombre = "Luis"; // ✗ Error
```

! REGLA DE ORO: Usa const por defecto. Usa let únicamente cuando sepas que necesitas reasignar la variable (ej: contadores).

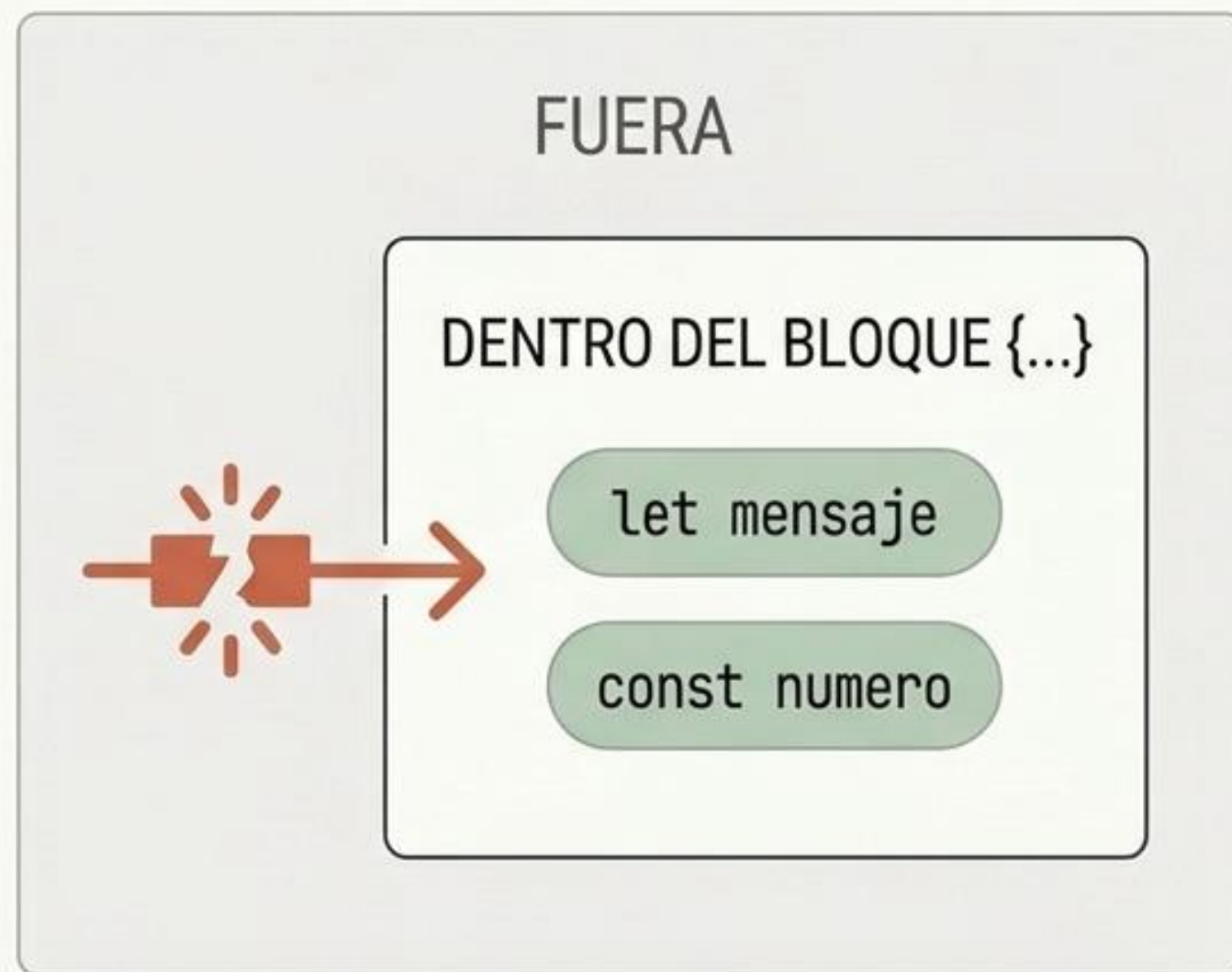
Matriz de Comparación: El adiós a var

Declaración	¿Reassignable?	Ámbito (Scope)	Uso Recomendado
var	Sí	Función	❌ Obsoleto
let	Sí	Bloque	⚠️ Solo si muta
const	No	Bloque	✅ Por defecto

let y const proporcionan un comportamiento predecible, fundamental para la inmutabilidad y gestión del estado en frameworks modernos.

Visualizando el Ámbito de Bloque (Scope)

```
if (true) {  
  let mensaje = "Hola";  
  const numero = 10;  
}  
console.log(mensaje); // ✗ ReferenceError
```



Takeaway: Lo que pasa en el bloque, se queda en el bloque.

Mapeando la Realidad (Tipos Primitivos)

String

"Ana"

(Texto)

Number

20, 10.5

(Enteros y
decimales usan
el mismo tipo)

Boolean

true, false

(Lógica binaria)

Undefined

Valor por
defecto (variable
declarada sin
asignar).

Null

Ausencia
intencional y
voluntaria de
valor.

> Nota de herramientas: Usa el operador '`typeof variable`' en consola para inspeccionar estos tipos en tiempo real.

Peculiaridades Históricas (Quirks)

⚠ Anomalía 1: El Falso Objeto

```
typeof null // "object"
```

Diagnosis: Es un **error histórico de diseño** en JavaScript. No significa que debamos tratar a null como un **objeto normal** con propiedades.

⚠ Anomalía 2: El Número que no es Número

```
const res = Number("hola");  
console.log(typeof res); // "number" (NaN)
```

Diagnosis: **NaN** (Not a Number) representa una **operación matemática fallida**, pero irónicamente pertenece a la categoría **técnica numérica**.

El Arsenal de Operadores

Aritméticos

+

-

*

/

```
** (Potencia: 2 ** 3 = 8)
```

```
% (Módulo - Crucial para  
pares/impares): 8 % 2 // 0
```

Comparación

>

<

>=

<=

Lógicos

&& (AND - Y)

|| (OR - O)

! (NOT - Negación)

```
edad >= 18 && tieneEntrada // true
```

Precedencia de los Operadores

Precedencia	Nombre	Signo
...	...	...
14	suma unaria	+
14	negación unaria	-
13	exponenciación	**
12	multiplicación	*
12	división	/
11	suma	+
11	resta	-
...	...	...
2	asignación	=
...	...	...

Operadores lógicos en JS

```
console.log("Hola" && 25); //?  
console.log("Hola" && 0);  //?  
console.log(10 && "DWECC"); //?
```

?

?

?

Con &&, JavaScript evalúa de izquierda a derecha y devuelve el primer valor falsy que encuentre. Si todos son truthy, devuelve el último valor.

Con || sucede lo contrario: evalúa de izquierda a derecha y devuelve el primer valor truthy que encuentre. Si todos son falsy, devuelve el último.

La Tabla Diagnóstica de Igualdad

== (Igualdad Débil)

Convierte los tipos en secreto antes de comparar.

```
0 == false // true  
"" == false // true
```

❌ Peligroso e impredecible.

=== (Igualdad Estricta)

Compara VALOR y TIPO de forma absoluta.

```
0 === false // false  
"" === false // false
```

✅ Seguro.

REGLA DEFINITIVA: En este módulo, usaremos única y exclusivamente === y !==.

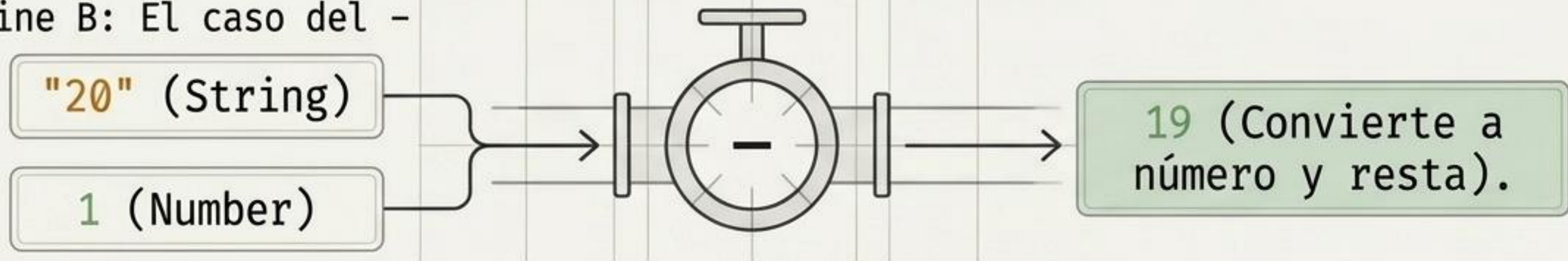
Coerción Implícita (La Magia Oscura)

Cuando JavaScript convierte automáticamente un valor de un tipo a otro para intentar resolver una operación.

Pipeline A: El caso del +



Pipeline B: El caso del -



⚠ Alerta: Confiar en esta magia genera bugs invisibles.

Coerción Explícita (Tomando el Control)

La solución a la magia oscura es declarar tu intención explícitamente usando funciones nativas.

Escudo Numérico

```
Number("20") → Garantiza un tipo number (o lanza NaN si es inválido).
```

Escudo de Texto

```
String(20) → Garantiza un tipo string.
```

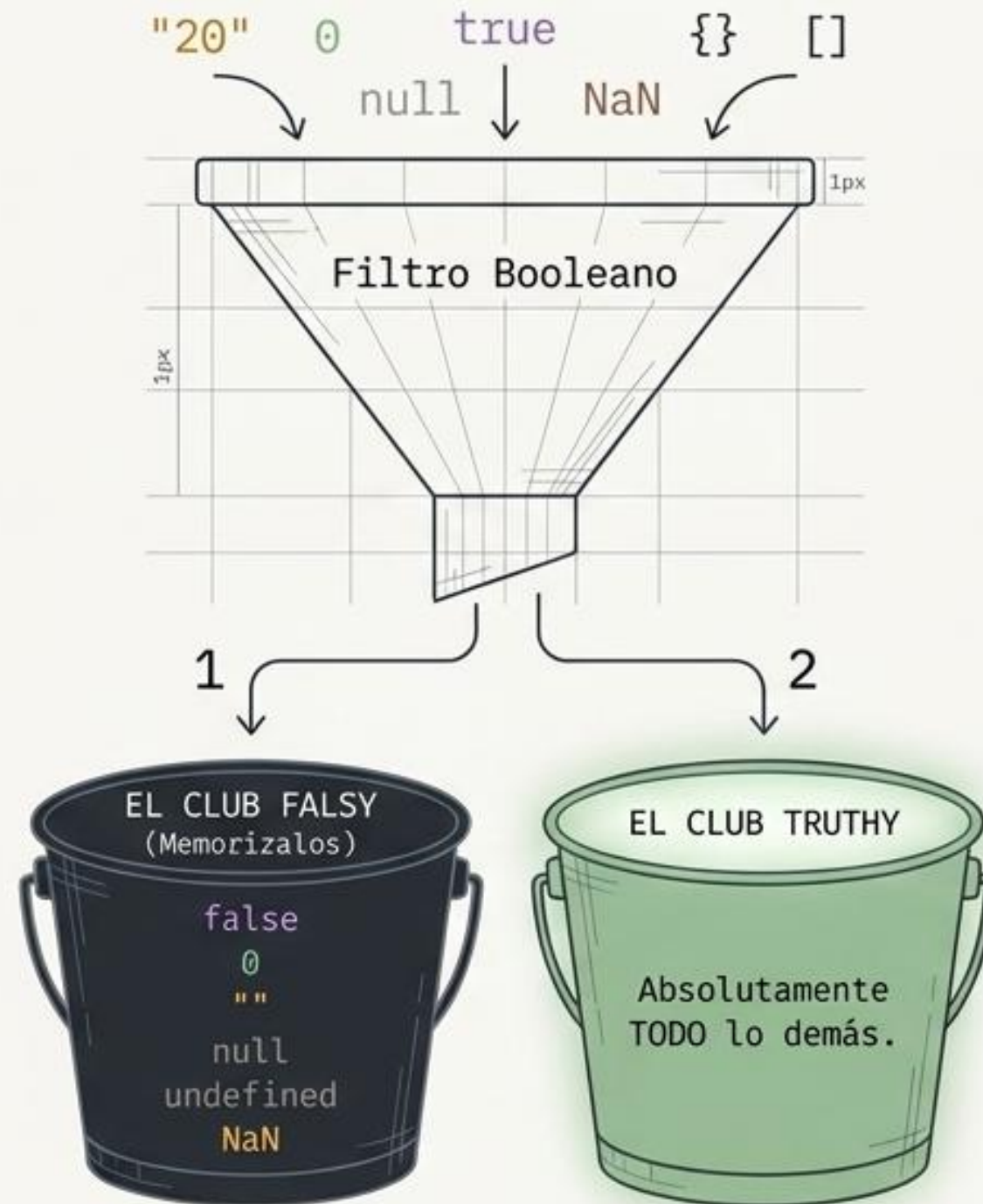
Escudo Lógico

```
Boolean(1) → Garantiza true o false.
```

Resolviendo la Slide 2: Por eso `Number(edad) + 1` (`21`) es la forma correcta y segura de sumar un input de texto.

El Embudo Condicional (Truthy vs Falsy)

En un `if()`, JavaScript evalúa los valores como verdaderos o falsos.



Ejemplo React-Ready

```
const nombre = "";  
if (nombre) { ... } → Al ser un string  
vacío, evalúa como Falsy y no se  
ejecuta.
```

Las 3 Reglas de Oro del JS Moderno

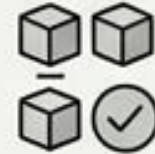
1. Declara con Intención



Usa `const` siempre por defecto. Cámbiate a `let` solo si sabes que la variable necesita mutar. Jamás uses `var`.



2. Compara con Rigor



Usa `===`. La igualdad débil (`==`) es un artefacto del pasado que genera errores invisibles.



3. Convierte sin Miedo



Domina la coerción explícita (`Number()`, `Boolean()`) y memoriza los 6 valores Falsy para tomar el control total del flujo de tu aplicación.



> **Siguiente Sesión:** Estructuras de control (Condicionales, bucles) y Arrow Functions...